

Realizzazione Server IRC  
Programmazione modulare, I/O Multiplexing e multithreading  
Laboratorio di Reti e Sistemi Distribuiti 2024-2025 - Unime

Giacomo Siracusa - 527084

2 settembre 2025

## Indice

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                             | <b>2</b>  |
| <b>2</b> | <b>Stato dell'arte</b>                          | <b>2</b>  |
| 2.1      | Protocollo IRC . . . . .                        | 2         |
| 2.2      | Comunicazione TCP Client/Server . . . . .       | 3         |
| 2.3      | I/O Multiplexing - select() . . . . .           | 4         |
| 2.4      | Multithreading . . . . .                        | 4         |
| 2.5      | Mutex e Condition Variable . . . . .            | 5         |
| 2.6      | Compilazione modulare . . . . .                 | 5         |
| <b>3</b> | <b>Implementazione Server</b>                   | <b>5</b>  |
| 3.1      | main() . . . . .                                | 6         |
| 3.2      | start_IRC_protocol() - Thread . . . . .         | 8         |
| 3.3      | nick() - per la creazione dell'utente . . . . . | 9         |
| 3.4      | join() . . . . .                                | 9         |
| 3.5      | privmsg() . . . . .                             | 10        |
| <b>4</b> | <b>Imp. client.c</b>                            | <b>11</b> |
| <b>5</b> | <b>Risultati</b>                                | <b>12</b> |
| <b>6</b> | <b>Conclusioni</b>                              | <b>14</b> |

# 1 Introduzione

Ho realizzato un singolo server IRC (Internet Relay Chat) concorrente in C, che gestisce le richieste: NICK, JOIN, PART, PRIVMSG, TOPIC e QUIT, da parte dei client connessi ad esso. Durante lo sviluppo mi sono servito della seguente documentazione: RFC1459, che definisce all'effettivo il protocollo IRC (<https://datatracker.ietf.org/doc/html/rfc1459>).

Sia il client che il server sono programmi modulari e multithread. Per ottenere mutua esclusione vengono utilizzati i Mutex, inoltre per controllare la creazione dei thread e la loro attesa, dispongo delle variabili di condizione.

Sul server, i thread che si occupano di elaborare le specifiche richieste, lavorano su due strutture dati dinamiche condivise: lista di utenti e lista di canali, dove per ogni canale vi è una lista dinamica per identificare i vari utenti che ne fanno parte. Sempre su lato server, la gestione delle connessioni TCP, viene eseguita tramite la chiamata di sistema `select()` che fornisce l'I/O multiplexing, garantendo anche la compatibilità su sistemi operativi che non fanno parte della famiglia GNU/Linux.

Per la manipolazione delle stringhe si è utilizzata la libreria `string.h`.

## 2 Stato dell'arte

Di seguito sono riportati dei concetti base, che senza di essi non avrei potuto garantire un'efficiente realizzazione del sistema. Riguardo al protocollo IRC, dato che, il server può comunicare solo con i client e non con altri server che si collegano successivamente, farò solo riferimento alle comunicazioni Client/Server.

### 2.1 Protocollo IRC

Lo sviluppo del protocollo IRC è stato ufficialmente riconosciuto con la pubblicazione della specifica RFC1459 nel 1993. Di preciso, dopo 4 anni da quando è stato implementato per la prima volta, come mezzo per chattare tra gli utenti di una BBS. Ora, il protocollo supporta una rete mondiale di server e client. Tutto è basato su testo, con il client che può essere qualsiasi programma socket in grado di connettersi al server.

Quindi, il protocollo IRC (Internet Relay Chat) è stato progettato per consentire conferenze basate su testo. Viene applicato comunemente su sistemi che utilizzano il protocollo di rete TCP/IP su architetture Client/Server. È adatto a funzionare su molte macchine in modo distribuito. Una configurazione tipica coinvolge un singolo processo (il server) che forma un punto centrale, in cui i client possono connettersi, eseguendo la consegna/multiplexing dei messaggi e altre funzioni.

Ogni client si distingue da tutti gli altri, da un nickname di massimo 9 caratteri e lo imposta inviando al server il comando NICK seguito dal nominativo. Un client, si può distinguere, oltre al nickname, anche per un altro concetto, ovvero, se è un operatore o meno di alcuni canali. L'operatore serve a mantenere un ragionevole livello di ordine all'interno dei canali, potendo: configurare le impostazioni di un canale, stabilire chi può essere membro e tanto altro. Tuttavia per questo progetto li definisco semplicemente, e servono solo per impostare l'argomento del canale (tramite il comando TOPIC).

Un canale è un gruppo denominato di uno o più client che riceveranno tutti i messaggi indirizzati a quel canale. Il canale viene creato implicitamente quando il primo client vi si unisce, e cessa di esistere quando l'ultimo client lo abbandona. Finché un canale esiste, qualsiasi client può farvi riferimento utilizzando il nome del canale. I nomi dei canali sono stringhe (che iniziano con il carattere '#') di lunghezza massima di 200 caratteri. A parte il requisito che il primo carattere sia '#', l'unica restrizione sul nome di un canale è che non può contenere (come i nickname): spazi (' ') o virgole (','), viene utilizzata dal protocollo come separatore di elementi di lista).

Per creare un nuovo canale o entrare a far parte di un canale esistente, un utente deve iscriversi (JOIN) al canale. Se il canale non esiste prima dell'iscrizione, il canale viene creato e l'utente che lo crea diventa un operatore del canale. Se il canale esiste già, l'accettazione o meno della richiesta di ISCRIVERSI a quel canale dipende dalle modalità correnti del canale. Ad esempio, se il canale è solo su invito, è possibile iscriversi solo se invitati. Come previsto dal protocollo, un utente può far parte di più canali contemporaneamente.

I messaggi IRC sono sempre righe di caratteri terminate da una coppia CR-LF (Carriage Return - Line Feed) e non devono superare la lunghezza di 512 caratteri, contando tutti i caratteri, compreso il

CR-LF finale. Pertanto, sono consentiti al massimo 510 caratteri per il comando e i suoi parametri. Rappresentazione BNF dei messaggi:

```
<message> ::= <command> <params> <crLf>
<command> ::= <letter> { <letter> }
<SPACE>    ::= ' ' { ' ' }
<params>    ::= <SPACE> [ ':' <trailing> | <middle> <params> ]
<middle>    ::= <Qualsiasi sequenza di caratteri, escluso lo spazio, NUL, CR e LF, il
    primo carattere non deve essere ':'>
<trailing>  ::= <Qualsiasi sequenza di caratteri, escluso NUL, CR e LF>
<crLf>     ::= CR LF
```

<Trailing> è solo un trucco sintattico per consentire lo spazio all'interno del parametro. Come si può notare sopra, trailing inizia per ':' in modo da riconoscerlo. Serve per spedire il testo effettivo.

Ulteriore sintassi:

```
<target>    ::= <to> [ ", " <target> ]
<to>        ::= <channel> | <nick>
<channel>   ::= '#' <chstring>
<nick>      ::= <letter> { <letter> | <number> | <special> }
<chstring>  ::= <any 8bit code except SPACE, NUL, CR, LF and comma (',')>
```

La maggior parte dei messaggi inviati al server genera una risposta di qualche tipo. La risposta più comune è la risposta numerica, utilizzata sia per gli errori che per le risposte normali. Una risposta numerica è come un normale messaggio, tranne per il fatto che la parola chiave è composta da tre cifre numeriche anziché da una stringa di lettere. Alcune risposte comuni:

- 421, significa che il comando inviato è sconosciuto;
- 451, client non registrato;
- 461, parametri insufficienti.

Oltre ai comandi NICK, JOIN e TOPIC a cui ho già fatto riferimento, ce ne sono altri 3 fondamentali che devono essere implementati:

- PRIVMSG, per inviare messaggi privati tra utenti o tra canali;
- PART, consente di uscire da uno o più canali;
- QUIT, chiude la connessione.

L'obiettivo principale di IRC è quello di fornire un forum che permetta conferenze facili ed efficienti (conversazioni uno a molti). IRC offre due mezzi per raggiungere questo obiettivo: specificando un solo canale oppure specificando liste di destinazioni composte da nick e/o canali.

## 2.2 Comunicazione TCP Client/Server

In sistemi linux, per trasmettere dati in rete da un'applicazione ad un'altra (presenti anche nella stessa macchina), si fa uso dell'interfaccia di rete denominata Linux Socket. Essa fornisce diverse funzioni chiave tra cui: creazione dei socket, definizione del tipo di comunicazione e invio/ricezione dati.

Il socket (in italiano "presa") è il punto terminale di una comunicazione, dove partono i messaggi scritti da un processo, verso un secondo end-point (socket). Ogni socket viene identificato da una coppia di valori: indirizzo IP, dell'host in rete e un numero di porta, che identifica diversi processi su un host.

Dato che in Linux tutto è un file, il socket è un particolare file speciale, simile ad una pipe. Per effettuare delle operazioni di I/O, esso, prima di tutto li deve identificare e lo fa tramite i loro file descriptor, valori numeri univoci restituiti al processo nel momento delle creazioni dei file.

La creazione di un socket viene eseguita con la chiamata di socket(), resa disponibile includendo la libreria netinet/in.h.

La funzione alloca in memoria il socket, inizializza il protocollo specifico (es: TCP, UDP, ecc.) e restituisce il file descriptor. Sia i processi client che il server devono eseguire questa funzione. I client, dovranno poi connettersi tramite la funzione connect(), dato l'utilizzo del protocollo TCP che richiede l'instauramento di una connessione, specificando l'indirizzo del server e la porta con cui si mette in ascolto il processo server. Essa però la potranno chiamare solamente dopo che il server avrà fatto il

binding del socket (associazione indirizzo IP e porta al socket), si sarà messo in ascolto e avrà chiamato la funzione `accept()` che accetta una singola connessione. Ogni connessione, il server la gestisce con un nuovo socket in cui si possono eseguire le seguenti funzioni: `read` e `write` rispettivamente lettura e scrittura dei messaggi ricevuti e inviati dal/al client.

## 2.3 I/O Multiplexing - `select()`

Si definisce chiamata bloccante, la chiamata di una funzione di I/O eseguita su un file descriptor (un socket per esempio) che sospende l'esecuzione del programma finché non viene completata. Durante l'attesa, il thread o il processo che ha invocato la chiamata rimane inattivo ("dormiente"), e lo scheduler passa il controllo della CPU ad altri processi/thread. Quando l'evento atteso si verifica, il sistema operativo riattiva il thread, la funzione riprenderà e dunque anche il programma. Normalmente, un esempio di chiamate bloccanti sono: `accept()` e `read()`. Se un server le lasciasse così, potrà eseguire una sola comunicazione alla volta. In pratica se un client sta comunicando, il server dunque ha una connessione stabilita dove vengono trasmessi messaggi. Se volesse connettersi un secondo client dovrà attendere, per instaurare il canale TCP e l'invio dei messaggi. L'attesa durerà fino a quando la connessione non termina e il server chiama nuovamente `accept()`.

Per rendere le chiamate non bloccanti su un determinato file descriptor, si passa il flag `O_NONBLOCK` come parametro nella system call `fcntl`. In questo modo le funzioni di lettura o scrittura eseguite sul file che si sarebbero bloccate ritornano immediatamente, evitando che il server resti bloccato.

Tuttavia adesso serve un modo per capire quando il server deve chiamare effettivamente le funzioni, ovvero quando ci sono nuovi dati da leggere. Una modalità è l'I/O multiplexing, per la gestione dell'I/O su file speciali che consente di tenerne sotto controllo molteplici allo stesso istante, permettendo di bloccare un processo per un tempo stabilito quando le operazioni di lettura o scrittura non sono immediatamente effettuabili (evitando attesa attiva), e di riprenderne l'esecuzione una volta che almeno una di quelle che erano state richieste diventi possibile, in modo da poterla eseguire con la sicurezza di non restare bloccati.

Ciò può essere implementato con la chiamata di sistema `select()` che interagisce direttamente con il kernel per monitorare lo stato di multipli file descriptor (FD). Il kernel mantiene per ogni FD una lista di processi in attesa di eventi (es: lista dei processi in attesa di dati su un socket). Quando dati arrivano su un socket, il kernel itera la lista dei processi in attesa e li risveglia. Il processo quindi dovrà semplicemente controllare i fd se sono pronti per l'operazione di I/O. Questo porta ad avere un costo computazionale  $\theta(n)$  dove  $n$  è il numero di fd del processo.

## 2.4 Multithreading

Un server concorrente, è quel server che smista tutte le richieste dei client ad altri thread o processi che le elaborano fornendo i messaggi di rispost. Evitando così di lasciare inattivo il processo che deve ricevere richieste di connessioni e messaggi. Situazione che può capitare invece in server iterativi, quando l'elaborazione di alcune risposte è molto complessa o in generale occupa molto tempo.

Un thread è l'unità di base d'uso della CPU e comprende un identificatore di thread (ID), un contatore di programma, un insieme di registri e una pila (stack). Simile a un PCB (Process Control Block), il thread infatti nasce come risoluzione di alcuni problemi rilevati nella gestione dei processi in un sistema multiprocessore, che permette, grazie alla concorrenza, l'esecuzione di processi diversi in parallelo su più core.

Il problema si riscontra quando si hanno più programmi in esecuzione con stesso codice sorgente contemporaneamente, c'è sicuramente uno spreco di memoria per via delle varie copie dei file sorgente, dei dati globali e anche molto probabilmente di alcune risorse.

La soluzione è quindi la condivisione di queste informazioni in modo da ridurre lo spazio in memoria e facilitare la gestione delle risorse in comune.

In pratica un thread è un determinato flusso di istruzioni (funzione) di un processo e per fare un'analogia con un programma in C, un thread è la funzione `main` che può accedere a variabili e a file globali.

Quando si hanno più threads che lavorano con dati globali uguali, si avrà il famoso concetto del multithreading. In C per creare e gestire un processo multithread si utilizza la libreria di GNU con l'header `pthread.h`.

## 2.5 Mutex e Condition Variable

Per evitare race condition (gara tra thread per chi esegue prima blocchi di istruzioni su dati condivisi) si deve far uso di meccanismi di sincronizzazione che si basano sul concetto di Lock. L'accesso a una sezione critica (blocco di istruzioni su dati condivisi) da parte di un processo viene bloccato da un altro processo. Quello che già sta eseguendo la sua sezione critica, prima di iniziare, ha messo un Lock che lo toglierà nel momento in cui esce dalla sua sezione critica. Così facendo si evita l'incostanza dei dati e si ottiene mutua esclusione. Il meccanismo che fa ciò si chiama Mutex. Esso si basa sulla coppia di istruzioni atomiche (che non possono essere interrotte) test-and-set sul lucchetto (un dato di tipo int). Se è 1 il lucchetto si può accedere alla sezione critica e si setta a 0, alla fine della sezione si setterà ad 1. Quindi se ad un thread il mutex risulta a 0 dovrà attendere.

Le variabili di condizione forniscono un'ulteriore metodologia per la sincronizzazione dei thread. Mentre i mutex implementano la sincronizzazione controllando l'accesso dei thread ai dati, le variabili di condizione consentono ai thread di sincronizzarsi in base al valore effettivo dei dati condivisi. Un esempio di utilizzo è l'esecuzione contemporanea di un numero limite massimo di thread.

Senza le variabili di condizione, il programmatore dovrebbe continuamente eseguire un controllo del contenuto di una risorsa condivisa dai thread per verificare se la condizione è soddisfatta. Questo può essere molto dispendioso in termini di risorse poiché il thread sarebbe continuamente impegnato in questa attività. Inoltre garantiscono l'atomicità delle istruzioni, quindi anche la consistenza dei dati, trasferendo il lock da un thread ad un altro. Per fare ciò, una variabile di condizione viene sempre utilizzata in congiunzione con un mutex.

Entrambi per utilizzarli basta includere la libreria pthread.h

## 2.6 Compilazione modulare

Il concetto "definizione di una variabile (o funzione)" indica il momento in cui una variabile o funzione viene creata, ovvero viene allocato il suo spazio in memoria. D'altro canto la **dichiarazione** informa il compilatore dell'esistenza e del tipo di una variabile o funzione, ma non necessariamente ne crea l'allocazione.

Grazie a ciò, in un file, si possono utilizzare risorse definite (allocate) su un altro file. La sintassi è la seguente:

```
extern int a; // Dichiarazione di una variabile senza definizione
int funzione(int x); // Dichiarazione di una funzione senza definizione
```

Ciò ovviamente non basta. Per funzionare, il compilatore deve effettuare una **compilazione modulare** ovvero deve avere a disposizione oltre al file contenente le dichiarazioni esterne, il file che definisce la variabile e la specifica funzione.

## 3 Implementazione Server

L'implementazione del server è compresa nei file:

- server.c; dove risiede il main e tutte le istruzioni che consentono di creare il server TCP, ponendolo in ascolto di nuove connessioni e messaggi da parte dei client. Ogni messaggio che riceve, lo passa come argomento ad un thread che si occupa di identificare il tipo di comando (NICK, JOIN, PRIVMSG...) e chiamare la corretta funzione per gestirla.
- service\_server.c, contiene le definizioni delle funzioni che gestiscono i vari comandi IRC, inviando in seguito delle risposte direttamente ai client. In esso sono definiti necessariamente i dati strutturati globali, di tipo **users** e **channels**:

```
//dati globali in service_server.c
users u = {0};
channels c = {0};
pthread_mutex_t mutex=PTHREAD_MUTEX_INITIALIZER;
```

Di seguito i corrispettivi campi:

```
//service_server.h
...
```

```

typedef struct {
    int sockfd;
    char nickname[SIZE_NICKNAME];
}user;

typedef struct {
    user *array;
    int size;
}users;

typedef struct {
    char name[SIZE_CHANNEL_NAME];
    int fd_operator;
    int *fd_members;
    int n_members;
    char topic[SIZE_TOPIC];
}channel;

typedef struct {
    channel *array;
    int size;
}channels;
...

```

- `utils_service_server.c`, risiedono tutte le sotto-funzioni utilizzate dalle funzioni in `service_server.c`. Lavora con le stesse strutture dati globali dichiarate esterne, così non c'è bisogno di passarle come argomenti.
- Infine, `utils_server.c`, definisce tutte le funzioni di utilità comuni che un server dovrebbe avere. In particolare un server che fa uso della `select()`.

Prima di vedere il codice di `server.c`, informo che l'esecuzione concorrente dei comandi IRC viene sincronizzata con un mutex diverso da quello che usa il `main`, per la variabile di condizione. Quest'ultima analizzata proprio adesso.

Nello specifico le variabili globali utilizzate nel `server.c` sono:

```

pthread_cond_t cond=PTHREAD_COND_INITIALIZER;
pthread_mutex_t lock= PTHREAD_MUTEX_INITIALIZER;
int contatore_threads;

```

In pratica, è solamente una condition variable che lavora con *contatore\_thread*, informando al `main` quanti thread stanno eseguendo e se esso deve mettersi in stato di sleep nel momento in cui il contatore raggiunge `MAX_NUM` (macro definita in `server.h`) . Ad ogni thread creato dunque si incrementa `contatore_thread`, garantendo mutua esclusione con il mutex *lock*.

Il file `server.h` presenta varie macro, dichiarazioni di funzioni di `utils_server` e `service_server.c` ed in fine la definizione del tipo di dato *message*. Questo tipo di dato strutturato conterrà un file descriptor socket associato ad un client e il messaggio che esso ha inviato al server. Ogni dato di tipo *message* viene allocato dinamicamente dal `main`, inizializzato e passato il suo indirizzo come argomento del thread che se ne occuperà. Sarà responsabilità di quest'ultimo liberare la memoria una volta finita l'elaborazione del comando IRC.

### 3.1 main()

Definizioni:

```

int main() {
    int array_sockfd[MAX_SOCKET];
    int size_array=0, max_sockfd=0;
    struct sockaddr_in serv_addr, cli_addr;
    socklen_t clilen = sizeof(cli_addr);
    pthread_t thread; // utilizzato alle chiamate di pthread_create()
    fd_set read_fds; // insieme di file descriptor che il kernel deve monitorare
    struct timeval tv;
    bool turn_off=false; // server acceso/spento
    ...
}

```

Per tenere traccia dei socket utilizzo *array\_socket*. La variabile *size\_array* indica il numero di socket effettivi, utile per la gestione di essi, e *max\_sockfd*, un file descriptor che serve alla select(). I dati strutturati di tipo *sockaddr\_in* contengono indirizzo IP e porta di ascolto del server e del client, utili per la comunicazione. La variabile *tv* serve a settare l'intervallo di tempo massimo, che il processo deve attendere dopo la select() per continuare l'esecuzione.

Prime istruzioni:

```
array_sockfd[0] = init_socket_TCP(&serv_addr, MAX_NUM, PORT);
size_array++;
max_sockfd = array_sockfd[0];

set_nonblocking(array_sockfd[0]);
...
```

Di seguito un elenco in ordine di alcune chiamate a funzioni principali: select, read, lock e unlock del mutex, pthread\_cond\_wait e pthread\_create. Continuo del main():

```
printf("Server multithread in ascolto su porta %d...\n", PORT);
while (!turn_off) {
    init_fd_set(&read_fds, array_sockfd, size_array);
    // set timeout
    tv.tv_sec= 15;
    tv.tv_usec = 0;
    int ready_sockfd = select(max_sockfd+1, &read_fds, NULL, NULL, &tv);
    if (ready_sockfd == -1) {
        perror("select failed");
        exit(EXIT_FAILURE);
    } else {
        //ACCEPTING SOCKET
        if (FD_ISSET(array_sockfd[0], &read_fds) && size_array < MAX_SOCKET) {
            int new_sockfd=accept_new_connection_by_select(array_sockfd, &
size_array, &cli_addr, &cliilen);

            if (new_sockfd > max_sockfd)
                max_sockfd = new_sockfd;
        }
        //READING SOCKETS
        if (size_array>1) {
            int i=1, j=0; // i serve a spazzolare tutti i fd, j tiene la conta
degli fd letti
            while (i<size_array && j<ready_sockfd && !turn_off) {
                if (FD_ISSET(array_sockfd[i], &read_fds)) {
                    message *m=malloc(sizeof(message)); //uso malloc dato che poi
deve essere passato al thread, e altrimenti si sovrascriverebbe
                    int n_bytes = read(array_sockfd[i], m->buffer, BUFFER_SIZE-1);
                    if (n_bytes >0) {
                        m->buffer[n_bytes]='\0';
                        if((strcmp(m->buffer, "server_off\r\n") == 0)) {
                            close_all_connections_by_select(array_sockfd,
size_array);

                            turn_off=true;
                            free(m);
                        } else {
                            m->sockfd = array_sockfd[i];
                            // attesa se il numero di threads in esecuzione e'
uguale a MAX_NUM

                            pthread_mutex_lock(&lock);
                            contatore_threads++;
                            while(contatore_threads>=MAX_NUM) {
                                pthread_cond_wait(&cond, &lock);
                            }
                            pthread_mutex_unlock(&lock);

                            if (!strncmp(m->buffer, "quit",4))
                                close_connection_by_select(array_sockfd, &
size_array, &i);

                            //Creazione thread!
                            pthread_create(&thread, NULL, start_IRC_protocol, m);
                        }
                    }
                }
            }
        }
    }
}
```

```

        }
        j++;
    }
    i++;
}
}
}

//attesa terminazione tutti i threads
pthread_mutex_lock(&lock);
while (contatore_threads>0) {
    pthread_cond_wait(&cond, &lock);
}
pthread_mutex_unlock(&lock);

close(array_sockfd[0]);
printf("Server off.\n");
return 0;

```

### 3.2 start\_IRC\_protocol() - Thread

Chiamata dal main, la seguente funzione, identifica il comando IRC presente nel messaggio arrivato in input. Controlla dunque, solo la prima sequenza di caratteri fino al primo spazio che trova, ed effettua una separazione. In base al comando chiamerà una funzione specifica di service\_server.c. Passando come argomenti il client socket e il resto del messaggio ancora non processato (i parametri). Tutto questo verrà eseguito in maniera concorrente al thread main. Una volta che la funzione di service\_server.c finisce, il messaggio di risposta sarà stato già inviato e si dovrà semplicemente liberare lo spazio di memoria di *m* e notificare tramite pthread\_cond\_signal che vi è un thread in meno in esecuzione.

```

void *start_IRC_protocol(void *arg) {
    message *m = (message *)arg;
    char *parameters=strchr(m->buffer, ' ');

    printf("FD: %d. Messaggio ricevuto: %s\n", m->sockfd, m->buffer);
    if (parameters!=NULL) {
        parameters[0]='\0';
        parameters++;
        if (parameters[strlen(parameters)-2]=='\r' && parameters[strlen(parameters)-1]=='\n') {
            parameters[strlen(parameters)-2]='\0';
            if (!strcmp(m->buffer, "nick"))
                nick(m->sockfd, parameters);
            else if (!strcmp(m->buffer, "join"))
                join(m->sockfd, parameters);
            else if (!strcmp(m->buffer, "part"))
                part(m->sockfd, parameters);
            else if (!strcmp(m->buffer, "topic"))
                topic(m->sockfd, parameters);
            else if (!strcmp(m->buffer, "quit"))
                quit(m->sockfd);
            else{
                privmsg(m->sockfd, m->buffer, parameters); //m->buffer contiene le
                destinazioni e params il testo.
            }
        }else {
            send_response(m->sockfd, "421 Comando sconosciuto");
        }
    }else {
        if (!strncmp(m->buffer, "quit",4))
            quit(m->sockfd);
        else
            send_response(m->sockfd, "421 Comando sconosciuto");
    }
    free(m); //deallocazione del messaggio dall'heap

    pthread_mutex_lock(&lock);
    contatore_threads--;
}

```



```

pthread_cond_signal(&cond);
pthread_mutex_unlock(&lock);

return NULL;
}

```

### 3.3 nick() - per la creazione dell'utente

```

void nick(int sockfd, char* nickname) {
    char buffer[BUFFER_SIZE-2]={0};
    if (strlen(nickname)>0) {
        if (strlen(nickname)>=SIZE_NICKNAME)
            nickname[SIZE_NICKNAME-1]='\0'; //riduco la lunghezza del nickname

        if (is_middle(nickname) && nickname[0]!='#') {
            pthread_mutex_lock(&mutex);

            user *p_u=get_user(sockfd);
            if (p_u!=NULL) { //se utente trovato
                if (get_user_by_nickname(nickname)!=NULL) { //se nickname esiste gia'
                    if (!strcmp(p_u->nickname, nickname)) //se l'utente ha gia' lo
                        stesso nickname
                        sprintf(buffer, "433 %s :Nickname gia' in uso", nickname);
                    else
                        sprintf(buffer, "436 %s :Collisione di nickname", nickname);

                    send_response(sockfd, buffer);
                }else
                    strcpy(p_u->nickname, nickname); //imposto il nuovo nickname
            }else { //creo un nuovo utente
                //alloco memoria
                if (u.size==0) {
                    u.array=malloc(sizeof(user));
                    u.size++;
                }else {
                    u.size++;
                    u.array=realloc(u.array,u.size*sizeof(user));
                }
                p_u=&u.array[u.size-1];
                //setto i dati del nuovo utente
                p_u->sockfd=sockfd;
                strcpy(p_u->nickname, nickname);
                printf("Utente creato: %s\n", p_u->nickname);
            }

            pthread_mutex_unlock(&mutex);
        }else {
            sprintf(buffer, "432 %s :Nickname non valido", nickname);
            send_response(sockfd, buffer);
        }
    }else
        send_response(sockfd, "432 nick :Nickname non dato");
}

```

### 3.4 join()

```

void join(int sockfd, char* channel_names) {
    bool fine=false;
    user *p_u=NULL;

    pthread_mutex_lock(&mutex);

    p_u=get_user(sockfd);
    if (p_u!=NULL) {
        do{
            char buffer[BUFFER_SIZE-2]={0};
            char *p_channel=channel_names;

```

```

        char *p_comma=strchr(channel_names,','); //puntamento alla prima virgola
di channel_names

        if (p_comma==NULL) {
            fine=true;
        }else {
            *p_comma='\0'; //cosi' p_channel si fermerà a p_comma
            channel_names=p_comma+1; //channel_names diventa tutto cio' che c'e'
dopo la prima virgola
        }
        if (strlen(p_channel)>1) {
            if (strlen(p_channel)>=SIZE_CHANNEL_NAME) // se il nome del canale
supera la lunghezza massima possibile
                p_channel[SIZE_CHANNEL_NAME-1]='\0'; //riduco la lunghezza
            if (p_channel[0]!='#') {
                p_channel++; //in modo da saltare '#'
                if (is_middle(p_channel)) {
                    channel *p_ch=get_channel(p_channel);
                    if (p_ch!=NULL) { // se e' stato trovato il canale
                        join_in_a_channel(p_ch, p_u, sockfd); //mi unisco
                    }else { //creo canale e mi unisco
                        c.size++;
                        if (c.size==1) {
                            c.array=malloc(sizeof(channel));
                        }else {
                            c.array=realloc(c.array,c.size*sizeof(channel));
                        }
                        channel *new_ch=&c.array[c.size-1];
                        // Inizializzo nuovo canale
                        strcpy(new_ch->name, p_channel);
                        new_ch->fd_operator=p_u->sockfd; //set operatore
                        new_ch->topic[0]='\0';
                        new_ch->n_members=0;
                        new_ch->fd_members=NULL;
                        printf("Canale creato %s - %s\n", p_channel, new_ch->name
);
                    }

                    join_in_a_channel(new_ch, p_u, sockfd);
                }
            }else {
                sprintf(buffer, "403 %s :Nome canale non valido", p_channel);
                send_response(sockfd, buffer);
            }
        }else {
            sprintf(buffer, "403 %s :Nome canale non valido", p_channel);
            send_response(sockfd, buffer);
        }
    }else {
        sprintf(buffer, "403 %s :Nome canale non valido", p_channel);
        send_response(sockfd, buffer);
    }
}
while(!fine);
}else {
    send_response(sockfd, "451 :Registrati tramite nick prima");
}
pthread_mutex_unlock(&mutex);
}

```

### 3.5 privmsg()

```

void privmsg(int sockfd, char* destinations, char* text) {
    char buffer[BUFFER_SIZE-2];
    bool fine=false;
    user *p_u;

    pthread_mutex_lock(&mutex);

    p_u=get_user(sockfd);
    if (is_trailing(text) && strlen(text)>1){ //se testo non vuoto ed e' valido

```

```

        text++;
        if (strlen(destinations)>0 && is_middle(destinations)) { //se destinations non
vuota ed e' valida
            if (p_u!=NULL) { //se utente registrato
                do {
                    //leggi una destinazione
                    char *dest=destinations;
                    destinations=strchr(destinations,',');
                    if (destinations==NULL)
                        fine=true;
                    else {
                        destinations[0]='\0';
                        destinations++;
                    }

                    if (dest[0]=='#') //se dest e' un canale
                        send_to_channel(++dest, p_u, text);
                    else
                        send_to_client(dest, p_u, text);
                }while (!fine);
            }else {
                sprintf(buffer,"451 privmsg :Non sei registrato");
                send_response(sockfd, buffer);
            }
        }else {
            sprintf(buffer,"411 privmsg :Nessun destinatario dato");
            send_response(sockfd, buffer);
        }
    }else {
        sprintf(buffer,"412 privmsg :Testo da mandare assente");
        send_response(sockfd, buffer);
    }
}

pthread_mutex_unlock(&mutex);
}

```

## 4 Imp. client.c

```

int sockfd;
bool quit;
pthread_mutex_t mutex=PTHREAD_MUTEX_INITIALIZER;

int main() {
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE]={0};
    pthread_t thread;

    sockfd = start_tcp_connection(&serv_addr);

    if (sockfd>0) {
        quit=false;

        printf("Connesso al server. Invia messaggi (scrivi 'help' per la guida):\n");

        pthread_create(&thread, NULL, read_IRC_replies, NULL);

        pthread_mutex_lock(&mutex);
        while (!quit) {
            pthread_mutex_unlock(&mutex);
            fgets(buffer, BUFFER_SIZE-2, stdin);

            if(strlen(buffer) > 0) {
                if(strcmp(buffer, "help\n") == 0)
                    print_help();
                else{
                    buffer[strlen(buffer)-1]='\r';
                    buffer[strlen(buffer)]='\n';
                }
            }
        }
    }
}

```

```

        send(sockfd, buffer, strlen(buffer), 0);

        if (strncmp(buffer, "quit", 4) == 0 || strcmp(buffer, "server_off\
r\n") == 0) {
            printf("Uscita...\n");
            pthread_mutex_lock(&mutex);
            quit=true;
            pthread_mutex_lock(&mutex);
        }
    }
    }
    memset(buffer, 0, BUFFER_SIZE);
}
pthread_mutex_unlock(&mutex);
close(sockfd);
}
return 0;
}

void *read_IRC_replies() {
    time_t t;
    char *c_time=NULL;
    char buffer[BUFFER_SIZE]= {0};

    pthread_mutex_lock(&mutex);
    while (!quit) {
        pthread_mutex_unlock(&mutex);
        int n = read(sockfd, buffer, BUFFER_SIZE);

        if (n > 0) {
            buffer[n-2] = '\0';
            if(strcmp(buffer, "quit") == 0) {
                printf("Uscita...\n");
                pthread_mutex_lock(&mutex);
                quit=true;
                pthread_mutex_unlock(&mutex);
            }else
                analyze_response(buffer);

            t=time(NULL);
            c_time=ctime(&t);
            c_time[strlen(c_time)-1]='\0';
            printf("%s: %s\n", c_time, buffer);
        }
    }
    pthread_mutex_unlock(&mutex);
    return NULL;
}

```

## 5 Risultati

Per avviare il server e i client, basta aver eseguito il comando make da terminale, dentro la cartella dei codici sorgente. In modo da compilare il tutto in maniera automatica e corretta. Dopo di che eseguire i file oggetto *server* e *client*.

Di seguito delle conferenze create con 4 client connessi contemporaneamente al server: Client 1

```

Connessione al server attiva. Invia messaggi (scrivi 'help' per la guida):
nick Chiara
join #serietv
Sat Aug 30 23:15:56 2025: 353 #serietv :Chiara
Sat Aug 30 23:15:56 2025: 331 #serietv :Nessun argomento presente
Sat Aug 30 23:16:33 2025: 301 #serietv - Aurelio :Ciao :)
Aurelio :Che serie tv mi consigli? *(messaggio privato)

```

Client 2:

```

join #serietv
Sat Aug 30 23:15:09 2025: 451 :Registrati tramite nick prima
nick Aurelio

```

```

join #serietv
Sat Aug 30 23:16:11 2025: 353 #serietv :Chiara,Aurelio
331 #serietv :Nessun argomento presente
#serietv :Ciao :)
Sat Aug 30 23:20:25 2025: 301 Chiara :Che serie tv mi consigli?

```

Client 3:

```

nick Giacomo
join #sport,#serietv
Sat Aug 30 23:17:06 2025: 353 #sport :Giacomo
331 #sport :Nessun argomento presente
353 #serietv :Chiara,Aurelio,Giacomo
331 #serietv :Nessun argomento presente
topic #sport :Notizie sportive
Sat Aug 30 23:17:39 2025: 332 #sport :Notizie sportive
#sport :Ma quanto e' forte Sinner?
Sat Aug 30 23:18:57 2025: 301 #sport - Laura :Troppo!
part #sport,#serietv

```

Client 4:

```

nick Laura
join #sport
Sat Aug 30 23:18:06 2025: 353 #sport :Giacomo,Laura
332 #sport :Notizie sportive
Sat Aug 30 23:18:41 2025: 301 #sport - Giacomo :Ma quanto e' forte Sinner?
#sport :Troppo!
quit
Uscita...

```

Server:

```

Server multithread in ascolto su porta 6677...
Nuovo client connesso, fd: 4.
FD: 4. Messaggio ricevuto: nick Chiara

```

```

Utente creato: Chiara
Nuovo client connesso, fd: 5.
FD: 5. Messaggio ricevuto: join #serietv

```

```

Send -> 451 :Registrati tramite nick prima
FD: 5. Messaggio ricevuto: nick Aurelio

```

```

Utente creato: Aurelio
FD: 4. Messaggio ricevuto: join #serietv

```

```

Canale creato #serietv - serietv
Send -> 353 #serietv :Chiara
Send -> 331 #serietv :Nessun argomento presente
FD: 5. Messaggio ricevuto: join #serietv

```

```

Send -> 353 #serietv :Chiara,Aurelio
Send -> 331 #serietv :Nessun argomento presente
FD: 5. Messaggio ricevuto: #serietv :Ciao :)

```

```

Send -> 301 #serietv - Aurelio :Ciao :)
Nuovo client connesso, fd: 6.
FD: 6. Messaggio ricevuto: nick Giacomo

```

```

Utente creato: Giacomo
FD: 6. Messaggio ricevuto: join #sport,#serietv

```

```

Canale creato #sport - sport
Send -> 353 #sport :Giacomo
Send -> 331 #sport :Nessun argomento presente
Send -> 353 #serietv :Chiara,Aurelio,Giacomo
Send -> 331 #serietv :Nessun argomento presente
FD: 6. Messaggio ricevuto: topic #sport :Notizie sportive

```

```

Send -> 332 #sport :Notizie sportive
Nuovo client connesso, fd: 7.
FD: 7. Messaggio ricevuto: nick Laura

```

```

Utente creato: Laura
FD: 7. Messaggio ricevuto: join #sport

Send -> 353 #sport :Giacomo,Laura
Send -> 332 #sport :Notizie sportive
FD: 6. Messaggio ricevuto: #sport :Ma quanto e' forte Sinner?

Send -> 301 #sport - Giacomo :Ma quanto e' forte Sinner?
FD: 7. Messaggio ricevuto: #sport :Troppo!

Send -> 301 #sport - Laura :Troppo!
FD: 4. Messaggio ricevuto: Aurelio :Che serie tv mi consigli?

Send -> 301 Chiara :Che serie tv mi consigli?
Client disconnesso. FD: 7
FD: 7. Messaggio ricevuto: quit

Utente 7 eliminato
FD: 6. Messaggio ricevuto: part #sport,#serietv

```

## 6 Conclusioni

Come si può notare dai programmi studiati e dai risultati ottenuti, ho realizzato un singolo server IRC concorrente in C, che gestisce richieste: NICK, JOIN, PART, PRIVMSG, TOPIC e QUIT, da parte dei client connessi ad esso. Servirsi della documentazione: RFC1459, che definisce il all'effettivo il protocollo IRC, in modo chiaro ed esaustivo, è stato davvero utile.

Penso di aver fatto un buon uso dei Mutex, delle variabili di condizione e delle strutture dati utilizzate. E di aver scelto la select come metodo ottimale per fornire I/O Multiplexing. Dato questo caso specifico iniziale, in cui si hanno pochi client da gestire alla volta e garantendo anche la compatibilità su sistemi operativi che non fanno parte della famiglia GNU/Linux.

Questo progetto, è solo una base di partenza su cui, in futuro, posso essere aggiunte diverse funzionalità come: il comando che configura le impostazioni dei singoli canali (se sono pubblici, privati, ...), l'autenticazione utente, cifratura dei messaggi spediti e la comunicazione tra server per ottenere maggiore disponibilità e capacità.